IPL AUCTION SYSTEM

DATABASE MANAGEMENT PROJECT

DONE BY-

527 - Aditi Hinge
528 - Vaishnavi Ingawale
531 - Shweta Katkar
537 - Saniya Mahalle

PROBLEM STATEMENT

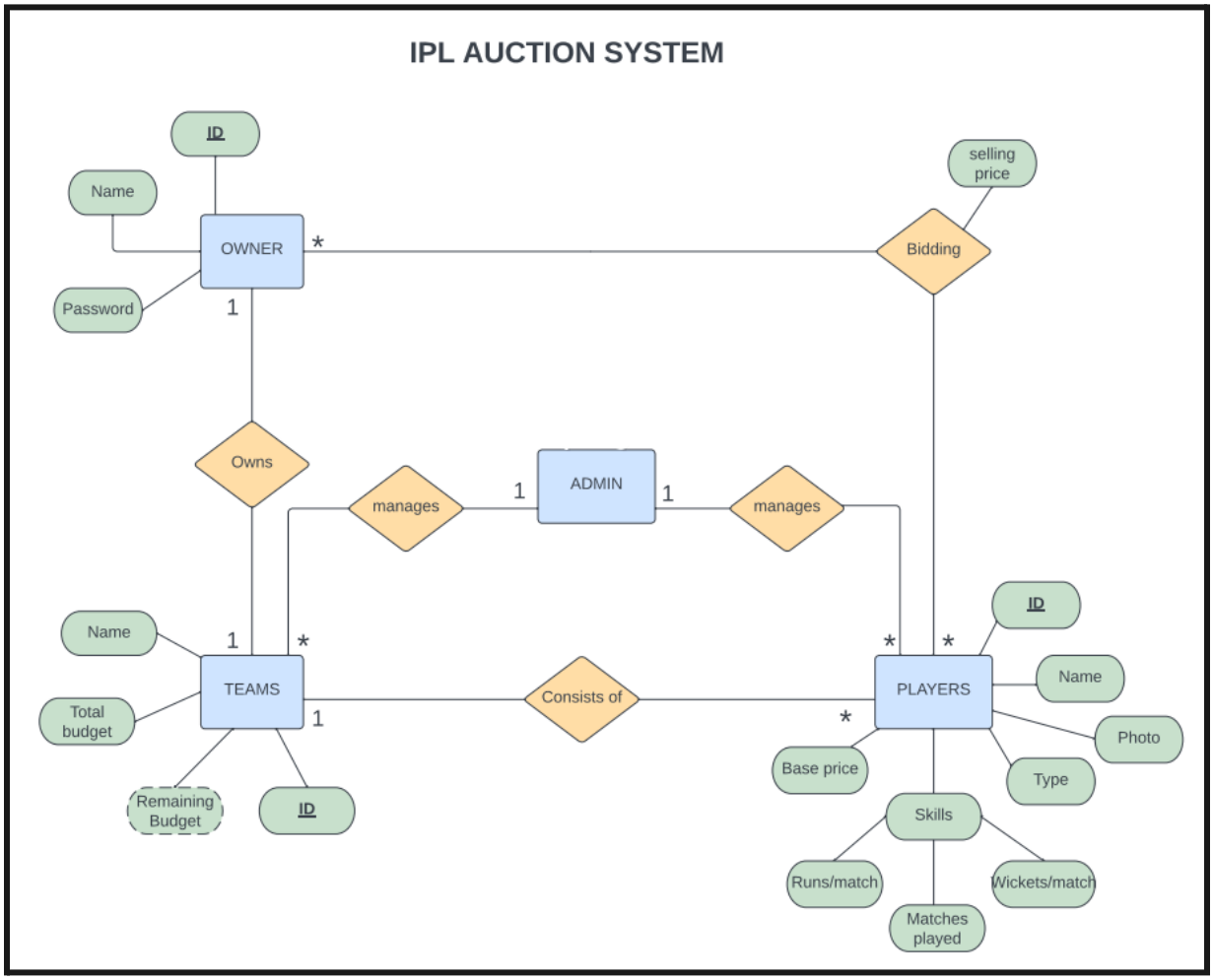
The IPL (Indian Premier League) is one of the most popular cricket leagues in the world, featuring some of the best cricket players from across the globe.The IPL auction is an integral part of the league where the franchises bid for players to form their team.

Design and develop a database for an IPL auction system using all the basic RDBMS concepts and execute all the CRUD operations. The database should be able to store and retrieve data related to the IPL auction process, including player information, franchise information, and auction results.

KEY FEATURES

- **Team creation:** Users can create their own cricket teams by entering team name, owner name, and owner email address.
- **Player management:** The system allows users to add, remove, and edit player details, such as name, age, playing position, and statistics.
- **Auction management:** The system provides a platform for conducting player auctions, where teams can buy the players they need.
- **Player statistics:** The system provides comprehensive statistics for each player, including their performance history, average runs, and wickets.
- **User-friendly interface:** The system has an intuitive and user-friendly interface, making it easy for users to navigate and manage their teams.

ENTITY-RELATION DIAGRAM



DATABASE REQUIREMENTS

- **Player table:** This table should store information about all the players who are up for auction, including their name, playing position, base price, nationality, and other relevant details like runs and matches.
- **Franchise table:** This table should store information about all the franchises participating in the auction, including their name, owner, budget and other relevant details.

- **Auction Result table:** This table should store information about the auction results, including the list of players who were sold, the final bid amount for each player, and the franchise that bought the player.

NORMALIZATION

- **First Normal Form (1NF)** is the first step in the normalization process in SQL. Specifically, a table is in first normal form if:

1. Each table cell contains a single, indivisible value (i.e., atomic value).
2. Each column in a table has a unique name.

Our tables are already in 1 NF.

- **Second Normal Form (2NF)** is the second step in the normalization process in SQL. Specifically, a table is in second normal form if:

1. It is already in first normal form (1NF).
2. All non-key columns in the table are dependent on the entire primary key.

Our tables are already in 2 NF.

- **Third Normal Form (3NF)** is the next step in the normalization process in SQL. Specifically, a table is in third normal form if:

1. It is already in second normal form (2NF).
2. All non-key columns in the table are dependent only on the primary key.

Our tables are already in 3 NF.

NORMALIZED TABLES

OWNER

ownerID	ownerName	ownerEmail	ownerPass
1	United Spirits	un@gmail.com	rcb
2	Ambani	MA@gmail.com	mi

TEAMS

teamID	teamName	totBudget
100	RCB	40
101	MI	40

OWNERSHIP

ownerID	teamID
1	100
2	101

PLAYERS

playerID	playerName	playerType	playerNation	playerImage	basePrice
111	Virat Kohli	Batter	Indian	PNG	15
222	Shane Watson	Bowler	Foreign	PNG	11

SKILLS

playerID	avgRuns	avgWickets	matches
111	56	2	234
222	23	5	109

SOLD PLAYERS (Relationship table)

playerID	teamID	sellingP
111	100	17
222	101	13

CREATE TABLES

1) Create OWNER table

```
create table owner(
   -> ownerID int primary key auto_increment,
   -> ownerName varchar(100) not null,
```

```
    -> ownerEmail varchar(100) not null unique,
    -> ownerPass varchar(10) not null);
Query OK, 0 rows affected (0.04 sec)
```

2) Create TEAMS table

```
 create table teams(
    -> teamID int primary key auto_increment,
    -> teamName varchar(100) not null unique,
    -> ownerID int not null unique,
    -> totBudget int not null default 40
    -> foreign key (ownerID) references owner(ownerID));
Query OK, 0 rows affected (0.04 sec)
```

3) Create PLAYERS table

```
create table players(
    -> playerID int primary key auto_increment,
    -> playerName varchar(100) not null,
    -> playerType varchar(50) not null,
    -> playerNation varchar(20) not null,
    -> basePrice int not null,
    -> playerAvail varchar(3) not null default "YES",
    -> playerImage longblob not null);
Query OK, 0 rows affected (0.02 sec)
```

4) Create OWNERSHIP table

```
create table ownership(
    -> teamID int,
    -> ownerID int,
    -> foreign key(teamID) references teams(teamID))
    -> primary key(ownerID, teamID));
Query OK, 0 rows affected (0.02 sec)
```

5) **Create SKILLS table**

```
create table skills(
   -> playerID int primary key auto_increment,
   -> avgRuns int not null,
   -> avgWickets int not null,
   -> matches int not null,
   -> foreign key(playerID) references players(playerID));
Query OK, 0 rows affected (0.02 sec)
```

6) **Create SOLD_PLAYERS table**

```
create table sold_players(
   -> playerID int not null,
   -> teamID int not null,
   -> sellingP int not null,
   -> primary key(playerID, teamID));
Query OK, 0 rows affected (0.03 sec)
```

PROCEDURES:

1) **makeBid Procedure** (procedure is called when a player is sold to a team)

```
create procedure makeBid(IN plrID int, IN franName varchar(100), IN bidAmt int)
   -> begin
   -> declare bp int;
   -> declare budget int;
   -> declare avail varchar(3);
   -> declare id int;
   -> declare maxP int;
   -> select playerAvail, basePrice into avail, bp from players where playerID =
plrID;
   -> select teamID, remBudget into id, budget from teams where teamName =
franName;
   -> select count(plrID) into maxP from sold_players where teamID = id;
   -> if maxP = 5 then
```

```
   -> signal sqlstate '45000' set message_text = "Players limit reached!";
   -> end if;
   -> if bidAmt < bp then
   -> signal sqlstate '45000' set message_text = "Invalid Bid Amount!!";
   -> end if;
   -> if bidAmt> budget then
   -> signal sqlstate '45000' set message_text = "Not Enough Budget!";
   -> end if;
   -> insert into sold_players values(plrID, id, bp, bidAmt);
   -> update players set playerAvail = "NO" where playerID = plrID;
   -> update teams set remBudget = remBudget - bidAmt where teamName =
franName;
   -> end //
Query OK, 0 rows affected (0.01 sec)
```

2) **releasePlayer** procedure

```
mysql> create procedure releasePlayer(IN plrID int)
   -> begin
   -> declare sp int;
   -> declare tmID int;
   -> select sellingP into sp from sold_players where playerID = plrID;
   -> select teamID into tmID from sold_players where playerID = plrID;
   -> update teams set remBudget = remBudget + 0.5*sp where teamID =
tmID;
   -> update players set playerAvail = "YES" where playerID = plrID;
   -> delete from sold_players where playerID = plrID;
   -> end //
```

QUERIES:

Search:

1) **Get the list of all players.**

```
Select playerName from players;
```

2) **Get the list of sold players.**

   Select playerName from players where playerAvail LIKE “NO”;

3) **Get the list of unsold players.**

   Select playerName from players where playerAvail LIKE “YES”;

4) **Get the list of players and the number of runs, wickets and matches played.**

   SELECT players.playerID, players.playerName, skills.avgRuns, skills.avgWickets, skills.matches FROM players INNER JOIN skills ON players.playerID = skills.playerID;

5) **Get the list of all players and their base prices.**

   Select playerID, playerName, basePrice from players;

6) **Get the list of all players who have been sold for a price greater than a certain amount.**
   SELECT players.playerID, players.playerName, players.basePrice , sold_players.sellingP FROM players INNER JOIN sold_players ON players.playerID = sold_players.playerID;

7) **Get the list of all players of a particular team.**

   Select (Select playerID, playerName, from players) from sold_players where teamName = “CSK”;

8) **Get the list of all Indian players.**

   Select playerID, playerName, playerNation from players where playerNation LIKE “INDIA”;

9) **Get the list of all foreign players.**

Select playerID, playerName, playerNation from players where playerNation NOT LIKE “INDIA”;

10) Get the list of all teams.

SELECT teamID, teamName from teams;

11) Get the list of all teams along with their owners and budget.

select ownership.ownerID, teams.teamName,owner.ownerName, teams.totBudget from ownership inner join owner on ownership.ownerID=owner.ownerID join teams on ownership.teamID = teams.teamID;

12) Get the list of all teams along with their total budget.

Select teamID, teamName, totBudget from teams;

13) Get the list of all teams along with their remaining budget.

select teams.totBudget - sum(sellingP) as remBudget, teams.teamName from sold_players inner join teams on sold_players.teamID = teams.teamID group by sold_players.teamID;

14) Get the list of all players who have been sold and the teams they have been sold to.

SELECT players.playerName, teams.teamName FROM players JOIN sold_players ON players.playerID = sold_players.playerID
JOIN teams ON sold_players.teamID = teams.teamID;

15) Get the list of all teams who have not bought any players

Select * from teams where totBudget == remBudget;

16) Get the list of players in the descending order of their base price.

```
Select * from players order by basePrice desc;
```

17) Get the list of players that are batters.

```
Select * from players where playerType = “Batter”
```

Update:

18) Update the base price of a player.

```
Create function updateBP(givenplayerID int, newPrice int)
begin
update players set basePrice = newPrice where playerID = givenplayerID;
end
```

19) Update achievements (number of matches/runs) of a player

```
Create function updateBP(newMatches int, newRuns int, newWickets int,
givenPlayerID int)
begin
update skills set avgRuns = newRuns, avgMatches = newMatches,
avgWickets = newWickets where playerID = givenplayerID;
end
```

20) Assigning a player to a team (to the team that wins the bid)

```
create procedure makeBid(IN plrID int, IN franName varchar(100), IN bidAmt int)
   -> begin
   -> declare bp int;
   -> declare budget int;
   -> declare avail varchar(3);
   -> declare id int;
   -> declare maxP int;
```

```
   -> select playerAvail, basePrice into avail, bp from players where playerID =
plrID;
   -> select teamID, remBudget into id, budget from teams where teamName =
franName;
   -> select count(plrID) into maxP from sold_players where teamID = id;
   -> if maxP = 5 then
   -> signal sqlstate '45000' set message_text = "Players limit reached!";
   -> end if;
   -> if bidAmt < bp then
   -> signal sqlstate '45000' set message_text = "Invalid Bid Amount!!";
   -> end if;
   -> if bidAmt> budget then
   -> signal sqlstate '45000' set message_text = "Not Enough Budget!";
   -> end if;
   -> insert into sold_players values(plrID, id, bp, bidAmt);
   -> update players set playerAvail = "NO" where playerID = plrID;
   -> update teams set remBudget = remBudget - bidAmt where teamName =
franName;
   -> end //
Query OK, 0 rows affected (0.01 sec)
```

Delete:

21) Remove a player from database.

Delete from players where playerID = givenPlayerID AND playerAvail = “YES”

22) Removing a player from a team.

```
Create function removePlayer(givenPlayerID int)
begin
DELETE FROM sold_players where playerID = givenplayerID;
Update players set playerAvail = “YES” where playerID = givenplayerID;
end
```

TOOLS USED:

1) Mysql 8.0
2) PYTHON: Flask-SqlAlchemy
3) Frontend - HTML, CSS (bootstrap template)